



Regression & Classification Projects in Machine Learning

Exploring predictive modeling through energy efficiency and loan approval applications



MACHINE LEARNING 2



NOVEMBER 16, 2025

Abdul Hannan (473578) • Stanisław Godlewski (473016)

University of Warsaw

Predicting Building Energy Efficiency

Project Objective

Develop models to accurately predict heating and cooling loads in residential buildings, enabling architects and engineers to design more energy-efficient structures. Accurate energy load prediction allows architects to simulate and optimize designs for more energy efficient buildings, saving costs and reducing carbon output.



Why Regression?

Energy consumption is measured in continuous values (kWh), making this a natural fit for regression techniques. Our models predict numerical outputs rather than categorical classifications.

Investigation

To investigate this issue, a dataset from Kaggle was utilized. This dataset looked into assessing the heating load and cooling load requirements of buildings as a function of building parameters. We perform energy analysis using 12 different building shapes simulated in Ecotect.

Bias

The dataset is generated from Ecotect simulations and assumes idealized conditions, which may not reflect real-world variability across geographic regions, building ages, or socioeconomic contexts.

Regression: Data & Approach

Dataset Characteristics

768

Observations

Building samples analyzed

8

Features

Architectural variables

2

Targets

Heating & cooling loads

Features: Relative_Compactness, Surface_Area, Wall_Area, Roof_Area, Overall_Height, Orientation, Glazing_Area, Glazing_Area_Distribution

Predict: Heating_Load, Cooling_Load

Methodology

- **Decision Tree Regressor**
A simple, interpretable model that splits data based on feature thresholds to predict continuous values. Prone to overfitting but useful as a baseline.
- **Random Forest Regressor**
An ensemble of many decision trees trained on random subsets of data and features. Reduces overfitting and improves generalization by averaging predictions.
- **Gradient Boosting Regressor**
Builds trees sequentially, where each new tree corrects errors from the previous one. Excels at capturing complex patterns and interactions in structured data—ideal for architectural energy modeling.
- **R^2**
Shows how much of the variation in energy load is explained by the model. Closer to 1 = better fit.
- **RMSE**
Measures average prediction error in the same units as the target (kWh). Penalizes large errors more heavily—critical when underestimating energy use could lead to design flaws.
- **MAE**
Gives the average magnitude of errors without squaring.
“On average, our prediction is off by X kWh.”
- **Data Split**
80/20 train-test split for robust validation

Regression Implementation

Our implementation demonstrates a clean, efficient approach to preparing data and training the best-performing model. Below are c

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Shape:", X_train.shape)

from sklearn.ensemble import GradientBoostingRegressor
gb_param = {
    "n_estimators": [100, 200],
    "max_depth": [3, 4, 5],
    "learning_rate": [0.05, 0.1, 0.2]
}

gb = GradientBoostingRegressor(random_state=42)
gb_cv = GridSearchCV(gb, gb_param, cv=5, scoring="neg_root_mean_squared_error")
gb_cv.fit(X_train, y_train)
```

This snippet showcases the essential steps: feature-target separation, stratified data splitting, and Gradient Boosting model initialization with 100 trees for optimal ensemble performance.

Further evaluation of the Gradient Boosting Model can be found on the next slide.

Gradient Boosting Model

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
```

```
def evaluate_model(model, X_test, y_test, name):
    pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    mae = mean_absolute_error(y_test, pred)
    r2 = r2_score(y_test, pred)
    print(f"{name} -> RMSE: {rmse:.3f}, MAE: {mae:.3f}, R²: {r2:.3f}")
    return {"Model": name, "RMSE": rmse, "MAE": mae, "R²": r2}
```

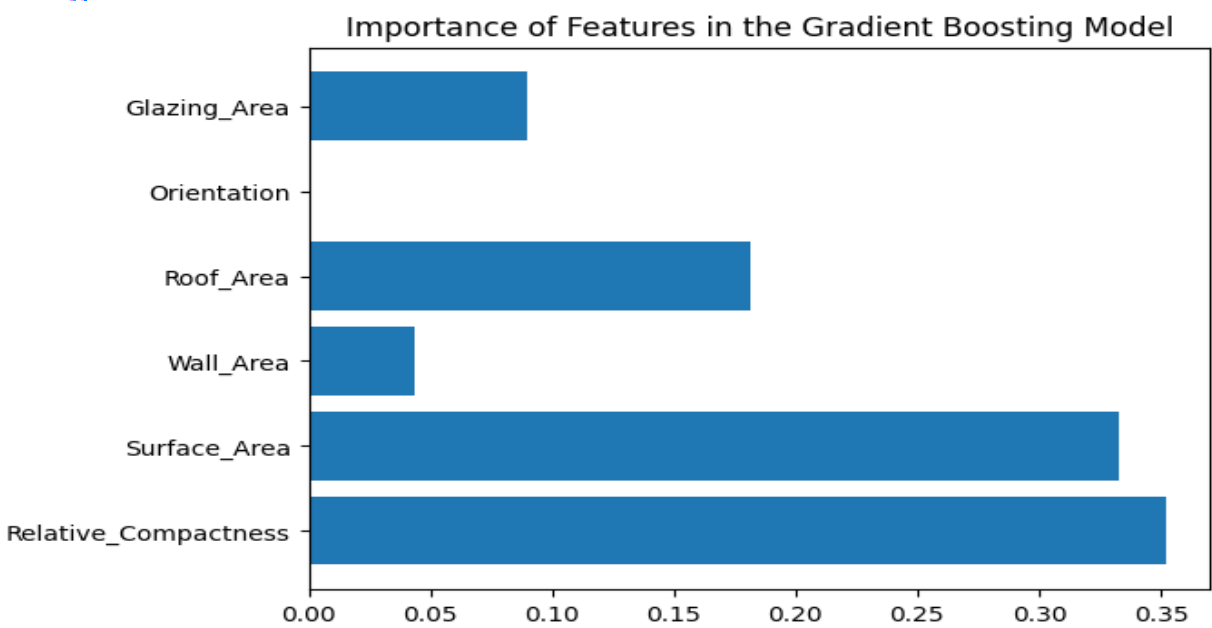
```
results = []
results.append(evaluate_model(dt_cv.best_estimator_, X_test, y_test, "Decision Tree"))
results.append(evaluate_model(rf_cv.best_estimator_, X_test, y_test, "Random Forest"))
results.append(evaluate_model(gb_cv.best_estimator_, X_test, y_test, "Gradient Boosting"))
```

	Model	RMSE	MAE	R²
0	Decision Tree	0.668432	0.468676	0.995713
1	Random Forest	0.841553	0.559012	0.993205
2	Gradient Boosting	0.523916	0.386365	0.997367

Gradient Boosting has the lowest RMSE, lowest MAE, and highest R2 out of all of the models. As such, the Gradient Boosting Model was utilized to capture the importance of the variables for heating loads in buildings.

```
import matplotlib.pyplot as plt

best_tree = gb_cv.best_estimator_
importances = best_tree.feature_importances_
plt.barh(features, importances)
plt.title("Importance of Features in the Gradient Boosting Model")
plt.show()
```



RMSE: 0.524 kWh
On average, the model's predictions are off by about 0.52 kilowatt-hours from the true heating load.

MAE: 0.386 kWh
The average absolute error is 0.39 kWh, meaning most predictions are within less than half a kWh of the actual value.

R²: 0.997
The model explains 99.7% of the variability in heating load based on building features. Relative_Compactness scored .35, accounting for 35% of the total feature importance in predicting the heating load.

Loan Approval Prediction System



Biases

Variables in this dataset such as education, income, and home ownership may encode systemic disparities potentially leading to biased outcomes against historically disadvantaged groups.

Project Objective

Predict whether a loan application should be approved (1) or rejected (0) based on applicant financial profiles and credit history. By relying on objective patterns rather than subjective judgment, these models help financial institutions serve more applicants responsibly, without compromising on risk control or ethical standards.

Why Classification?

Classification enables automated, data-driven decisions in high-stakes domains like finance. By accurately predicting whether a loan applicant is likely to repay, institutions can reduce default risk, expand access to credit, and minimize human bias.

Investigation

To investigate this issue, a dataset from Kaggle was utilized. This dataset is a synthetic version inspired by the original Credit Risk dataset on Kaggle and enriched with additional variables based on Financial Risk for Loan Approval data. SMOTENC was used to simulate new data points to enlarge the instances.

Classification: Data & Approach

Dataset Overview

45K

Observations

Loan applications

14

Features

Financial indicators

1

Target
Loan Status

Features: person_age, person_gender, person_education, person_income, person_emp_exp, person_home_ownership, loan_amnt, loan_intent, loan_int_rate, loan_percent_income, cb_person_cred_hist_length, credit_score, previous_loan_defaults_on_file

Predict: loan_status

Class Imbalance: Handled using
`class_weight="balanced"` to prevent model bias
toward majority class

Methodology

•Logistic Regression

A linear baseline model that estimates the probability of loan approval using input features. Interpretable and efficient for binary outcomes.

•Random Forest

An ensemble of decision trees that reduces overfitting by averaging predictions across diverse subsets of data and features. Handles non-linear patterns well.

•XGBoost

A powerful gradient boosting algorithm that builds trees sequentially to correct previous errors. Highly effective with structured tabular data and robust to class imbalance when properly configured.

•AUC

Shows how well the model tells apart approved vs. rejected loans. Closer to 1 = better separation.

•Accuracy

% of all loan predictions that are correct.

•Precision

How often the model correctly rejects applicants
High precision = few good applicants wrongly denied.

•Recall

How often the model incorrectly accents applicants
High recall = fewer bad loans slipping through.

•Data Split

80/20 train-test split for robust validation

Classification Implementation

Our implementation demonstrates a clean, efficient approach to preparing data and training the best-performing model.

On the right side are code snippets of splitting the data for testing and from the preparation of the XGBoost Classifier.

This snippet prepares the dataset for modeling by separating its features, encoding categorical variables, splitting the data with stratification, and utilizing an XGBooster. The XGBooster tunes hyperparameters using GridSearchCV with 5-fold cross-validation and ROC-AUC scoring to optimize model performance.

Further Evalaution of XGBoost Classifier can be found on the next slide.

```
y = df["loan_status"]
X = df.drop("loan_status", axis=1)

X_encoded = pd.get_dummies(X, columns=X.select_dtypes(include=['object']).columns, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42, stratify=y
)

print("Shape:", X_train.shape)

xgb_param = {
    "max_depth": [3, 5],
    "learning_rate": [0.1],
    "n_estimators": [100],
    "gamma": [0]
}
xgb = XGBClassifier(random_state=42, eval_metric="logloss", use_label_encoder=False)
xgb_cv = GridSearchCV(xgb, xgb_param, cv=5, scoring="roc_auc")
xgb_cv.fit(X_train, y_train)
```

XGBoosting Model

```
def evaluate_classification(name, model, X_test, y_test):
    pred = model.predict(X_test)
    pred_proba = model.predict_proba(X_test)[:, 1]
    auc = roc_auc_score(y_test, pred_proba)
    acc = accuracy_score(y_test, pred)
    print(f"\n{name}")
    print(f"→ AUC: {auc:.3f}, Accuracy: {acc:.3f}")
    print(classification_report(y_test, pred))

    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f"{name} - Confusion Matrix")
    plt.ylabel("True")
    plt.xlabel("Predicted")
    plt.show()

    return {"Model": name, "AUC": auc, "Accuracy": acc}

eval_results = []
eval_results.append(evaluate_classification("Logistic Regression", lr_cv.best_estimator_, X_test, y_test))
eval_results.append(evaluate_classification("Random Forest", rfc_cv.best_estimator_, X_test, y_test))
eval_results.append(evaluate_classification("XGBoost", xgb_cv.best_estimator_, X_test, y_test))

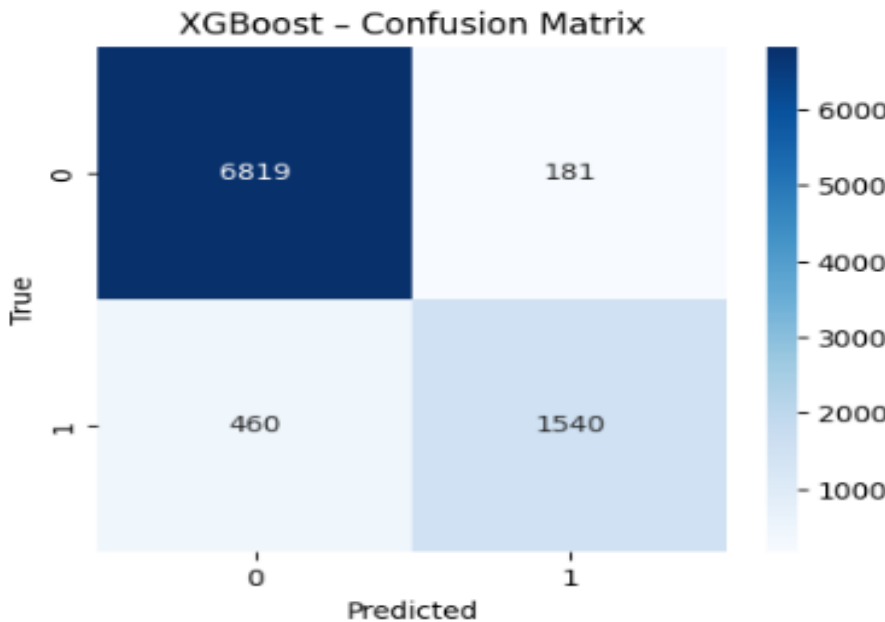
results_df = pd.DataFrame(eval_results)
print("\n Summary Table:")
print(results_df)
```

GridSearchCV enabled systematic exploration of regularization strength while addressing class imbalance. XGBoost had the strongest results. As such, it was chosen to be evaluated due to having the highest accuracy and AUC out of all of the models.

.

XGBoost
→ AUC: 0.975, Accuracy: 0.929

	precision	recall	f1-score	support
0	0.94	0.97	0.96	7000
1	0.89	0.77	0.83	2000
accuracy			0.93	9000
macro avg	0.92	0.87	0.89	9000
weighted avg	0.93	0.93	0.93	9000



Summary Table:

	Model	AUC	Accuracy
0	Logistic Regression	0.780286	0.794222
1	Random Forest	0.971383	0.894889
2	XGBoost	0.975499	0.928778

AUC: 0.975
The model can distinguish between borrowers who will default and those who won't with a 0.975 confidence rating.
Accuracy: 92.9%
The model correctly classifies over 92% of all loan applicants.
Precision (default class): 89%
When the model predicts a loan will default, it is correct 89% of the time.
Recall (default class): 77%
The model identifies 77% of all actual default cases.

Conclusion:

- Gradient Boosting is highly effective for energy modeling—captures complex interactions between building features and energy use, enabling fast, reliable estimates for sustainable design.
- In energy modeling, gradient boosting uncovers complex, nonlinear relationships between building features. This allows architects and engineers to generate fast, accurate predictions that inform smarter, greener building decisions.
- XGBoost is ideal for loan approval systems—handles imbalanced data well, resists overfitting, and delivers stable, interpretable decisions critical in finance.
- XGBoost allows those who are in need of a loan to obtain a loan with confidence.
- Both models show how choosing the right algorithm for the domain leads to real-world impact.